R Bootcamp: Answer Key

Every code answer below was run in R 4.3; the `##` lines are its real output. Random results depend on `set.seed`, so your simulated numbers will differ a little, but they should be close to the exact values given. Many drills have other correct answers; if yours runs and gives the right number, it's right.

Module 0: Setup and the R console

Module 1: Python → R: the rewiring

1.1 PREDICT

```
x = c(10, 20, 30, 40, 50)
x[2]
x[-2]
x[c(1, 5)]
x[length(x) - 1]
x[2:3] * 2
```

```
## [1] 20
## [1] 10 30 40 50
## [1] 10 50
## [1] 40
## [1] 40 60
```

1.2 PREDICT

```
n = 5
1:n-1
1:(n-1)
2*1:3
0:4/2
```

```
## [1] 0 1 2 3 4
## [1] 1 2 3 4
## [1] 2 4 6
## [1] 0.0 0.5 1.0 1.5 2.0
```

1.3 PREDICT

```
10 %% 3
10 %/% 3
2^10
rep(c(1, 2), 3)
rep(c(1, 2), each = 3)
rep(c(0, 1), c(2, 3))
seq(2, 11, 3)
```

```
## [1] 1
## [1] 3
## [1] 1024
## [1] 1 2 1 2 1 2
## [1] 1 1 1 2 2 2
## [1] 0 0 1 1 1
## [1]  2  5  8 11
```

1.4 PREDICT

```
sum(1:100)
cumsum(1:5)
prod(5:1)
c(1, 2) + c(10, 20, 30, 40)
length(c(c(1, 2), 3:5))
```

```
## [1] 5050
## [1]  1  3  6 10 15
## [1] 120
## [1] 11 22 31 42
## [1] 5
```

1.5 WRITE

```
c(1, 1, 2, 4)
box = rep(c(0, 1), c(30000, 10000)); length(box); mean(box)
seq(0.05, 1, 0.05)
deck = rep(1:4, each = 13); table(deck)
```

```
## [1] 1 1 2 4
## [1] 40000
## [1] 0.25
##  [1] 0.05 0.10 0.15 0.20 0.25 0.30 0.35 0.40 0.45 0.50 0.55 0.60 0.65 0.70 0.75
## [16] 0.80 0.85 0.90 0.95 1.00
## deck
##  1  2  3  4
## 13 13 13 13
```

1.6 WRITE

```
sum((1:10)^2)
sum(1 / 2^(0:10))
```

```
## [1] 385
## [1] 1.999023
```

1.7 WRITE

The shortcut formula from Midterm 1 is exactly what you need here.

```
box = c(2, rep(5, 2), rep(7, 3), 9)
mean(box)
sqrt(mean(box^2) - mean(box)^2)    # shortcut formula: mean of squares minus
square of mean
```

```
## [1] 6
## [1] 2.070197
```

1.8 PREDICT

```
x = c(5, 6, 7)
x[c(TRUE, FALSE, TRUE)]
x[-c(1, 2)]
x[4] = 8; x
y = c(); y[3] = 1; y
1:0
```

```
## [1] 5 7
## [1] 7
## [1] 5 6 7 8
## [1] NA NA  1
## [1] 1 0
```

Logical indexing keeps the TRUE positions. Assigning past the end grows the vector and fills the gap with `NA`. `1:0` counts *down* to give `1 0`. That last one breaks `for (i in 1:n)` when `n = 0`; see Module 3.

Module 2: Logical vectors: the simulation engine

2.1 PREDICT

```
v = c(3, 8, 1, 8, 6)
v >= 6
sum(v >= 6)
mean(v == 8)
v[v < 5]
which(v == 8)
```

```
## [1] FALSE  TRUE FALSE  TRUE  TRUE
## [1] 3
## [1] 0.4
## [1] 3 1
## [1] 2 4
```

2.2 PREDICT

```
v = c(3, 8, 1, 8, 6)
any(v == 2); all(v > 0)
c(2, 8) %in% v
ifelse(v %% 2 == 0, "even", "odd")
sum(v[v > 2])
```

```
## [1] FALSE
## [1] TRUE
## [1] FALSE  TRUE
## [1] "odd"  "even" "odd"  "even" "even"
## [1] 25
```

2.3 PREDICT

```
u = c(0.10, 0.25, 0.60, 0.80)
0*(u < 1/4) + 4*(u >= 1/4 & u < 3/4) + 8*(u >= 3/4)
1*(u <= 0.5)
```

```
## [1] 0 4 4 8
## [1] 1 1 0 0
```

The first line maps `u` onto the box {0, 4, 4, 8} (Lecture 13): P(0) = 1/4, P(4) = 1/2, P(8) = 1/4. The second line is a fair coin, 1 = heads (the Lecture 14 `cointoss`).

2.4 WRITE

```
sum(1:1000 %% 3 == 0 | 1:1000 %% 5 == 0)
mean(1:6 >= 5)
sum((1:100)[1:100 %% 2 == 0])
```

```
## [1] 467
## [1] 0.3333333
## [1] 2550
```

2.5 WRITE

(Run on 20 made-up pairs, so the numbers are rough.) For (c), filter first, *then* compare: `mean(rolls[coin == 1] >= 5)`. Writing `mean(rolls >= 5 & coin == 1)` estimates the *joint* probability P(A and B), not P(A | B).

```
mean(rolls == 6)
mean(rolls %% 2 == 0 & coin == 1)
mean(rolls[coin == 1] >= 5)
mean(rolls[coin == 0])
```

```
## [1] 0.15
## [1] 0.25
## [1] 0.4545455
## [1] 3.111111
```

2.6 WRITE

```
u = c(0.1, 0.3, 0.6, 0.8)
1*(u < 1/4) + 2*(u >= 1/4 & u < 3/4) + 3*(u >= 3/4)
```

```
## [1] 1 2 2 3
```

2.7 DEBUG

(b) and (c) are correct; they're the same computation. (a) estimates the joint P(sum = 7 and first = 3), which is 1/36. (d) is wrong: P(B)/P(A) is not P(B | A). The correct ratio is P(A and B)/P(A), and (c) computes exactly that. The true value is 1/6.

Module 3: Functions, if/else, loops

3.1 PREDICT

```
f = function(x) {
  if (x > 3) {
    return(x * 2)
  } else {
    return(x + 1)
  }
}
f(2)
f(5)
sapply(1:5, f)
```

```
## [1] 3
## [1] 10
## [1]  2  3  4  8 10
```

3.2 PREDICT

```
s = 0
for (k in 2:5) { s = s + k }
s
p = 1
for (k in c(2, 3, 4)) { p = p * k }
p
v = c()
for (k in 1:3) { v = c(v, k, -k) }
v
```

```
## [1] 14
## [1] 24
## [1]  1 -1  2 -2  3 -3
```

3.3 PREDICT

```
count = 0
x = 100
while (x > 1) {
  x = x / 2
  count = count + 1
}
c(count, x)
```

```
## [1] 7.00000 0.78125
```

3.4 WRITE

One-line alternatives: `return(1*(runif(1) <= p))` or `sample(c(1, 0), 1, prob = c(p, 1 - p))`.

```
cointoss = function(p) {
  u = runif(1)
  if (u <= p) {
    return(1)
  } else {
    return(0)
  }
}
set.seed(2); c(cointoss(0.5), cointoss(0.9), cointoss(0.1))
```

```
## [1] 1 1 0
```

3.5 WRITE

```
MAXABS = function(v) {
  best = 0
  for (x in v) {
    a = x
    if (a < 0) { a = -a }
    if (a > best) { best = a }
  }
  return(best)
}
MAXABS(c(2, 9, 5, -4, 0)); MAXABS(c(-12, 3))
```

```
## [1] 9
## [1] 12
```

3.6 WRITE

```
num_heads = function(n) {
  heads = 0
  for (i in 1:n) {
    heads = heads + cointoss(0.5)
  }
  return(heads)
}
num_heads2 = function(n) sum(sample(c(0, 1), n, replace = TRUE))
set.seed(3); num_heads(10); num_heads2(10)
```

```
## [1] 5
## [1] 7
```

3.7 DEBUG

(1) `while (roll = 6)` should be `while (roll != 6)`: `=` assigns, and the logic is backwards. (2) `sample(1:6)` returns a whole shuffled vector; it needs `sample(1:6, 1)`. (3) `n + 1` computes a value and throws it away; it must be `n = n + 1`. (4) `return n` needs parentheses: `return(n)`. Fixed version:

```
first_six = function() {
  n = 0
  roll = 0
  while (roll != 6) {
    roll = sample(1:6, 1)
    n = n + 1
  }
  return(n)
}
set.seed(4); first_six()
```

```
## [1] 6
```

3.8 WRITE

```
runs_until = function(k) {
  streak = 0
  tosses = 0
  while (streak < k) {
    tosses = tosses + 1
    if (runif(1) <= 0.5) { streak = streak + 1 } else { streak = 0 }
  }
  return(tosses)
}
set.seed(5)
sims = numeric(100000)
for (i in 1:100000) { sims[i] = runs_until(2) }
mean(sims)
```

```
## [1] 5.97266
```

3.9 PREDICT

```
g = function(n) {
  if (n <= 1) {
    return(1)
  }
  return(n * g(n - 1))
}
g(5)
```

```
## [1] 120
```

It's recursion: $n!$. R has `factorial(n)` built in (Module 4).

Module 4: Exact probabilities: counting, binomial, hypergeometric, normal

4.1 TRANSLATE

```
dbinom(3, 10, 0.5)
```

```
## [1] 0.1171875
```

4.2 TRANSLATE

```
1 - pbinom(6, 10, 0.5)
```

```
## [1] 0.171875
```

4.3 TRANSLATE

```
1 - pbinom(7, 10, 0.5)
```

```
## [1] 0.0546875
```

4.4 TRANSLATE

```
pbinom(1, 12, 1/6)
```

```
## [1] 0.3813326
```

4.5 TRANSLATE

```
pbinom(70, 100, 0.65) - pbinom(59, 100, 0.65)
```

```
## [1] 0.7513791
```

4.6 TRANSLATE

```
dhyper(2, 13, 39, 5)
```

```
## [1] 0.2742797
```

4.7 TRANSLATE

```
1 - dhyper(0, 4, 48, 5)
```

```
## [1] 0.341158
```

4.8 TRANSLATE

```
phyper(2, 8, 10, 6)
```

```
## [1] 0.4366516
```

4.9 TRANSLATE

```
1 - phyper(3, 8, 92, 10)
```

```
## [1] 0.003047199
```

4.10 TRANSLATE

```
phyper(3, 8, 92, 10) - phyper(0, 8, 92, 10)
```

```
## [1] 0.5803995
```

4.11 TRANSLATE

```
c(1 - phyper(4, 10, 5, 10), phyper(5, 5, 10, 10))
```

```
## [1] 1 1
```

4.12 TRANSLATE

```
choose(11, 4)
```

```
## [1] 330
```

4.13 TRANSLATE

```
choose(14, 4) * choose(10, 7)
```

```
## [1] 120120
```

4.14 TRANSLATE

```
factorial(8) / factorial(5)
```

```
## [1] 336
```

4.15 TRANSLATE

```
pnorm(2) - pnorm(-1)
```

```
## [1] 0.8185946
```

4.16 TRANSLATE

```
c(qnorm(0.8, 64, 3), 1 - pnorm(70, 64, 3))
```

```
## [1] 66.52486370  0.02275013
```

4.17 TRANSLATE

```
1 - pnorm(59.5, 50, 5)
```

```
## [1] 0.02871656
```

4.18 TRANSLATE

```
dhyper(35, 350, 650, 100)
```

```
## [1] 0.0879135
```

4.19 TRANSLATE

```
dbinom(2, 9, 1/6) * (1/6)
```

```
## [1] 0.04651361
```

4.20 TRANSLATE

```
(1 - pbinom(1, 7, 1/6)) / (1 - pbinom(0, 7, 1/6))
```

```
## [1] 0.4580325
```

4.11: both are 1. With only 5 men, at least 5 of the 10 chosen must be women; this is the W25 Q4b trick. 4.19: the first 9 rolls must hold exactly 2 sixes, and then roll 10 must be a six.

4.21 DEBUG

(a) should be `1 - pbinom(59, 82, 0.78)`; 60 itself must be counted. (b) The third argument is the number of *non*-hearts, 39, not 52. (c) "at most" needs `pbinom`, not `dbinom`. (d) Drawing at once means without replacement, so it's hypergeometric: `dhyper(2, 12, 8, 5)`. (e) `pnorm` gives the *left* tail; you need `1 - pnorm(59.5, 50, 5)`.

4.22 PREDICT

```
dbinom(0:3, 3, 0.5)
pbinom(3, 3, 0.5)
dhyper(1, 1, 9, 3)
choose(5, 0) + choose(5, 5)
```

```
## [1] 0.125 0.375 0.375 0.125
## [1] 1
## [1] 0.3
## [1] 2
```

4.23 WRITE

Answer: 8, matching the W25 key. `choose(39, c)/choose(52, c)` is the same as the product $\frac{39}{52} \cdot \frac{38}{51} \cdots$.

```
c_vals = 1:10
p_more = choose(39, c_vals) / choose(52, c_vals)   # first c cards all non-spades
round(1 - p_more, 3)                               # P(need <= c cards)
min(c_vals[1 - p_more >= 0.9])
```

```
##  [1] 0.250 0.441 0.586 0.696 0.778 0.840 0.885 0.918 0.942 0.960
## [1] 8
```

Module 5: Randomness: runif, sample, set.seed

5.1 WRITE

```
set.seed(3)
sum(sample(1:6, 2, replace = TRUE))
1 * (runif(1) < 0.3); sample(c(1, 0), 1, prob = c(0.3, 0.7))
sample(c(2, 5, 5, 7, 7, 7, 9), 1)
sum(sample(c(0, 1), 3, replace = TRUE))
```

```
## [1] 7
## [1] 0
## [1] 0
## [1] 7
## [1] 2
```

5.2 PREDICT

(i) and (iii) only. (ii) is impossible: there's only one 4 and no replacement. (iv) is impossible: there's no 3. (v) is impossible: size = 3 always returns 3 values.

5.3 WRITE

```
deck = rep(1:4, each = 13)
is_flush = function() length(unique(sample(deck, 5))) == 1
set.seed(4); mean(replicate(200000, is_flush()))
4 * choose(13, 5) / choose(52, 5)
```

```
## [1] 0.00205
## [1] 0.001980792
```

5.4 DEBUG

(1) Needs `replace = TRUE`. (2) Should be `runif(1) < 0.3`. (3) Needs `replace = TRUE`: without it you can't draw 3 from a 2-element vector, and that's an error. (4) A hand is dealt *without* replacement, so drop `replace = TRUE`.

5.5 WRITE

```
sample_x = function(n) {
  p = runif(n)
  return(0*(p < 1/4) + 4*(p >= 1/4 & p < 3/4) + 8*(p >= 3/4))
}
set.seed(5); X = sample_x(1000000)
mean(X); sd(X) * sqrt((1000000 - 1) / 1000000)
```

```
## [1] 3.996632
## [1] 2.829313
```

Module 6: Simulation recipes: the core of the exam's R

6.1 WRITE

Appending with `c()` in a loop is slow for 10^5 runs (a few seconds), but it's the course style. `numeric(N)` with `results[i] =` is much faster.

```
set.seed(10)
results = c()
for (i in 1:100000) {
  s = sum(sample(1:6, 2, replace = TRUE))
  results = c(results, s == 7)
}
mean(results)       # exact 1/6
```

```
## [1] 0.16822
```

6.2 WRITE

```
simMax = function() max(sample(1:6, 2, replace = TRUE))
set.seed(11); N = 100000
ms = numeric(N); for (i in 1:N) ms[i] = simMax()
c(mean(ms), sd(ms) * sqrt((N - 1) / N))
all36 = outer(1:6, 1:6, pmax)          # 6x6 table of maxima
c(mean(all36), sqrt(mean(all36^2) - mean(all36)^2))   # exact: 161/36 = 4.47, SE
1.40
```

```
## [1] 4.475340 1.406055
## [1] 4.472222 1.404084
```

6.3 FILL

Blanks: `ones_seen < 2`; `n1 / (n0 + n1)`; `ones_seen + 1`; `n1 - 1`; `n0 - 1`; `draws`. Unlike W25's `draw1`, this version updates both counts, which is the general pattern. For a check: with 3 zeros and 2 ones, the expected position of the second 1 is 2·(5+1)/3 = 4.

```
draw_two_ones = function(n0, n1) {
  ones_seen = 0
  draws = 0
  while (ones_seen < 2) {
    p = n1 / (n0 + n1)
    draws = draws + 1
    if (runif(1) < p) {
      ones_seen = ones_seen + 1
      n1 = n1 - 1
    } else {
      n0 = n0 - 1
    }
  }
  return(draws)
}
set.seed(12); mean(replicate(100000, draw_two_ones(3, 2)))   # exact E = 4
```

```
## [1] 4.00228
```

6.4 WRITE

If you want the host's move simulated explicitly: `opened = setdiff(1:3, c(pick, car))`, and if that has two values, sample one. Be careful: `sample(x, 1)` with a length-1 numeric `x` samples from `1:x` (Module 5, trap 2). Use `x[sample(length(x), 1)]`.

```
monty_switch = function() {
  car = sample(1:3, 1)
  pick = sample(1:3, 1)
  return(1 * (pick != car))     # switching wins iff first pick was wrong
}
set.seed(13); mean(replicate(100000, monty_switch()))   # exact 2/3
```

```
## [1] 0.66799
```

6.5 WRITE

```
set.seed(14)
shared = replicate(100000, any(duplicated(sample(1:365, 23, replace = TRUE))))
mean(shared)
1 - prod((365 - 0:22) / 365)
```

```
## [1] 0.50826
## [1] 0.5072972
```

6.6 EXPLAIN

(a) P(the sum of three dice is at least 15). (b) The SE of the max of two tickets drawn without replacement from {1, 2, 2, 5}. (c) P(heads | roll = 6). The coin and die are independent, so it's about 0.5. (d) P(ahead after 100 $1 bets on red in roulette): +1 with probability 18/38, −1 with probability 20/38.

6.7 WRITE

```
set.seed(15); N = 100000
xs = numeric(N); ys = numeric(N)
for (i in 1:N) {
  t = sample(c(0, 4, 4, 8), 2)
  xs[i] = t[1]; ys[i] = t[2]
}
cov(xs, ys) * (N - 1) / N
t2 = replicate(N, sample(c(0, 4, 4, 8), 2, replace = TRUE))   # 2 x N matrix
cov(t2[1, ], t2[2, ]) * (N - 1) / N
```

```
## [1] -2.640447
## [1] -0.04030286
```

6.8 WRITE

```
set.seed(16)
net = replicate(100000, sum(sample(c(35, -1), 100, replace = TRUE, prob = c(1, 37)
/ 38)))
mean(net > 0)
1 - pbinom(2, 100, 1/38)       # ahead iff 36W > 100 iff W >= 3
```

```
## [1] 0.49036
## [1] 0.4915671
```

6.9 WRITE

The absolute error tends to grow (like $\sqrt{n}$). The percentage error tends to shrink (like $1/\sqrt{n}$). Any single run is noisy, which is why the lecture plots many values of n.

```
set.seed(17)
for (n in c(100, 1000, 10000, 100000)) {
  heads = sum(sample(c(0, 1), n, replace = TRUE))
  cat(n, abs(heads - n/2), round(abs(100 * heads / n - 50), 3), "\n")
}
```

```
## 100 1 1
## 1000 5 0.5
## 10000 50 0.5
## 1e+05 31 0.031
```

6.10 WRITE

Let $I_f = 1$ if face f appears at least once. The number of distinct faces is $\sum_f I_f$, and $E(I_f) = 1 - (5/6)^6$. Linearity of expectation works even though the I_f are dependent (Lecture 13: "$E(X + Y) = E(X) + E(Y)$ for any X and Y").

```
set.seed(18)
mean(replicate(100000, length(unique(sample(1:6, 6, replace = TRUE)))))
6 * (1 - (5/6)^6)
```

```
## [1] 3.99145
## [1] 3.990612
```

Module 7: The Gauntlet: timed, exam conditions

G1 TRANSLATE

```
1 - phyper(2, 9, 11, 6)
dbinom(3, 6, 9/20)
pbinom(2, 6, 9/20)
```

```
## [1] 0.5743034
## [1] 0.3032184
## [1] 0.4415177
```

G2 TRANSLATE

```
pbinom(55, 80, 0.6) - pbinom(44, 80, 0.6)
pnorm(55.5, 80 * 0.6, sqrt(80 * 0.6 * 0.4)) - pnorm(44.5, 80 * 0.6, sqrt(80 * 0.6
* 0.4))
```

```
## [1] 0.7467637
## [1] 0.7443036
```

G3 WRITE

```
simX = function() {
  N = sample(1:6, 1)
  draws = sample(c(0, 1, 1), N, replace = TRUE)
  return(sum(draws))
}
set.seed(19)
xs = c()
for (i in 1:10000) { xs = c(xs, simX()) }
mean(xs)
sd(xs) * sqrt(9999 / 10000)
```

```
## [1] 2.3085
## [1] 1.425106
```

G4 FILL

```
first_ace = function() {
  deck = sample(rep(c(0, 1), c(48, 4)))
  return(which(deck == 1)[1])
}
set.seed(20)
sims = numeric(50000)
for (i in 1:50000) {
  sims[i] = first_ace()
}
prob = mean(sims <= 5); prob
1 - choose(48, 5) / choose(52, 5)     # exact
```

```
## [1] 0.33736
## [1] 0.341158
```

G5 EXPLAIN

It estimates P(Z = 0), where X and Y are two draws without replacement from {1, 2, 2, 3} and Z = X(Y − 1) (Lecture 13's discussion question). Z = 0 exactly when Y = 1, and P(Y = 1) = 1/4. Note that `replicate` accepts a whole `{ }` block.

```
set.seed(21)
z = replicate(20000, { t = sample(c(1, 2, 2, 3), 2); t[1] * (t[2] - 1) })
mean(z == 0)
```

```
## [1] 0.251
```

G6 DEBUG

(1) `heads =` overwrites the whole vector; it should be `heads[i] = .` (2) `sample(c(0, 1), 10)` needs `replace = TRUE`. (3) The correction is upside down; it should be `sqrt(9999 / 10000)`.

```
set.seed(22)
heads = numeric(10000)
for (i in 1:10000) {
  heads[i] = sum(sample(c(0, 1), 10, replace = TRUE))
}
sd(heads) * sqrt(9999 / 10000)
```

```
## [1] 1.576353
```

G7 WRITE

```
set.seed(23); N = 1000000
user = runif(N) < 0.001
pos = ifelse(user, runif(N) < 0.98, runif(N) < 0.01)
mean(user[pos])
```

```
## [1] 0.08625704
```

G8 WRITE

```
cointoss_a = function(p) sample(c(1, 0), 1, prob = c(p, 1 - p))
cointoss_b = function(p) 1 * (runif(1) <= p)
set.seed(24); mean(replicate(10000, cointoss_a(0.3))); mean(replicate(10000,
cointoss_b(0.3)))
```

```
## [1] 0.2959
## [1] 0.3022
```

G9 EXPLAIN

It's the box {−3, −3, 1, 1, 1, 10}: P(−3) = 1/3, P(1) = 1/2, P(10) = 1/6. E(X) = −1 + 0.5 + 10/6 = 7/6 ≈ 1.17. E(X²) = 3 + 0.5 + 100/6 = 20.17, so SE = $\sqrt{20.17 - 1.36} \approx 4.34$.

```
box = c(-3, -3, 1, 1, 1, 10); c(mean(box), sqrt(mean(box^2) - mean(box)^2))
```

```
## [1] 1.166667 4.336537
```

G10 WRITE

The draws are without replacement, so they aren't independent, and the variance of the sum isn't n times the variance of the box. (The EV formula still works: 4 × 0.078 = 0.312.)

```
coins = c(rep(0.01, 3), rep(0.05, 2), rep(0.10, 4), 0.25)
set.seed(25)
tot = replicate(100000, sum(sample(coins, 4)))
c(mean(tot), sd(tot) * sqrt((100000 - 1) / 100000))
sqrt(4) * sqrt(mean(coins^2) - mean(coins)^2)     # formula (with replacement):
too big
```

```
## [1] 0.3119631 0.1123801
## [1] 0.1370547
```

Module 8: Flashcards: memorize until instant

#	Task	Code
1	Vector of 5 zeros	`numeric(5)`
2	The box {1, 1, 2, 4}	`c(1, 1, 2, 4)`
3	39 zeros then 13 ones	`rep(c(0, 1), c(39, 13))`
4	0, 0.25, 0.5, 0.75, 1	`seq(0, 1, 0.25)`
5	Last element of v	`v[length(v)]`
6	v without its first element	`v[-1]`
7	Elements of v above 3	`v[v > 3]`
8	How many elements of v equal 2	`sum(v == 2)`
9	Fraction of v that is positive	`mean(v > 0)`
10	Positions where v equals 1	`which(v == 1)`
11	First position where v equals 1	`which(v == 1)[1]`
12	Is 5 one of the values in v?	`5 %in% v`
13	10 mod 3	`10 %% 3`
14	5!	`factorial(5)`
15	10 choose 3	`choose(10, 3)`
16	Ordered choices of 3 from 10	`factorial(10) / factorial(7)`
17	P(X = 4), X ~ Bin(10, 0.3)	`dbinom(4, 10, 0.3)`
18	P(X ≤ 4), X ~ Bin(10, 0.3)	`pbinom(4, 10, 0.3)`
19	P(X ≥ 4), X ~ Bin(10, 0.3)	`1 - pbinom(3, 10, 0.3)`
20	P(X > 4), X ~ Bin(10, 0.3)	`1 - pbinom(4, 10, 0.3)`
21	P(2 ≤ X ≤ 5), X ~ Bin(10, 0.3)	`pbinom(5, 10, 0.3) - pbinom(1, 10, 0.3)`
22	P(exactly 2 red) drawing 5 w/o replacement from 7 red, 13 blue	`dhyper(2, 7, 13, 5)`
23	P(at most 2 red), same box and draws	`phyper(2, 7, 13, 5)`
24	Area left of z = 1.5	`pnorm(1.5)`
25	P(X > 80), X normal with mean 70, SD 8	`1 - pnorm(80, 70, 8)`
26	z for the 95th percentile	`qnorm(0.95)`
27	One uniform number on (0, 1)	`runif(1)`
28	1 with probability p, else 0	`1 * (runif(1) < p)`

29	One die roll	`sample(1:6, 1)`
30	Ten die rolls	`sample(1:6, 10, replace = TRUE)`
31	Shuffle the vector v	`sample(v)`
32	3 tickets w/o replacement from box	`sample(box, 3)`
33	Biased coin: 1 w.p. 0.3, 5 tosses	`sample(c(1, 0), 5, replace = TRUE, prob = c(0.3, 0.7))`
34	Function header with argument n	`f = function(n) { ... return(result) }`
35	Loop i from 1 to N	`for (i in 1:N) { ... }`
36	Loop while x is below 10	`while (x < 10) { ... }`
37	Append x to the vector res	`res = c(res, x)`
38	Estimate SE from N simulated values in sims	`sd(sims) * sqrt((N - 1) / N)`
39	Estimate P(A | B) from logical vectors A and B	`mean(A[B])`
40	Run expression 1000 times, keep results	`replicate(1000, expr)`

One-page summary (review before the exam)

Need	R
exactly / at most / at least k	`dbinom(k,n,p)` / `pbinom(k,n,p)` / `1-pbinom(k-1,n,p)`
fewer than / more than k	`pbinom(k-1,n,p)` / `1-pbinom(k,n,p)`
a to b inclusive	`pbinom(b,n,p)-pbinom(a-1,n,p)` or `sum(dbinom(a:b,n,p))`
without replacement	same patterns with `dhyper(x, m, n, k)`, `phyper`: x of m successes, n failures, k drawn
normal	`pnorm(x, mean, sd)` = area left; `qnorm(q, mean, sd)`; continuity: ±0.5
count / multiset	`factorial`, `choose`, perms `factorial(n)/factorial(n-k)`
random	`runif(n)`, `sample(x, size, replace=TRUE/FALSE, prob=)`, `set.seed`
simulate	one-run function, then `for (i in 1:N) res[i] = f()` or `replicate(N, f())`
summarize	P: `mean(event)` · EV: `mean(x)` · SE: `sd(x)*sqrt((N-1)/N)` · P(A|B): `mean(A[B])`
syntax traps	`return(x)` · `} else {` · `==` vs `=` · `1:n-1` · `sample` defaults to no replacement · `x[-1]` drops